



FORMULARIO DE PRODUCCIÓN DE CONTENIDOS

NOMBRE DE LA ASIGNATURA: Desarrollo en plataformas
NOMBRE COMPLETO DEL DOCENTE CONTENIDISTA: Damián Aníbal Nicolalde Rodríguez

DESARROLLO

1. INTRODUCCIÓN DE LA CLASE

En la evolución del desarrollo web, la interacción dinámica y la organización del código se han convertido en pilares fundamentales para construir aplicaciones mantenibles y eficientes. En este contexto, jQuery surgió como una de las bibliotecas más influyentes de JavaScript, simplificando la manipulación del DOM, la gestión de eventos y la creación de efectos visuales mediante una sintaxis más legible. Su importancia histórica radica en haber unificado la forma en que los desarrolladores controlaban la interfaz antes del auge de los frameworks modernos. En esta clase, el estudiante explorará cómo jQuery permite vincular la Vista con el Modelo a través del Controlador, estableciendo los cimientos del patrón MVC en el frontend.

Asimismo, se introduce el concepto de persistencia de datos del lado del cliente, donde el modelo asume la responsabilidad de acceder y representar información proveniente de fuentes locales como JSON, CSV, XML o SQLite (mediante sql.js), así como lecturas remotas desde Firebase Firestore. A través de ejemplos prácticos, se mostrará cómo la arquitectura MVC organiza la interacción entre las tres capas, garantizando independencia entre la lógica de negocio, la manipulación de la interfaz y el almacenamiento. Con esta comprensión, el estudiante podrá desarrollar aplicaciones web más estructuradas, interactivas y orientadas a datos, preparándose para el siguiente nivel: la conexión del modelo con servidores y servicios en la nube.

Clase 6: jQuery y transición a arquitectura MVC

RDA 1: Programar aplicaciones web dinámicas, interactivas y con diseño visualmente adaptable, que permita mejorar la experiencia del usuario.

Reto # 1

6.1 jQuery y transición a arquitectura MVC

Durante los primeros años del desarrollo web, trabajar directamente con JavaScript y el DOM era un proceso complejo, repetitivo y propenso a errores. Cada navegador tenía su propia forma de implementar eventos, propiedades y métodos, lo que obligaba a los desarrolladores a escribir múltiples versiones del mismo código. En este contexto surge



jQuery, una biblioteca de código abierto lanzada en 2006 por John Resig, que cambió radicalmente la manera en que se construían interfaces dinámicas.

Su propósito original —“Write less, do more” (“Escribe menos, haz más”)— sintetiza perfectamente su aporte: reducir la cantidad de código necesario para manipular el DOM, gestionar eventos, ejecutar animaciones y realizar peticiones AJAX, garantizando compatibilidad entre navegadores.

Con el tiempo, jQuery no solo se consolidó como una herramienta práctica, sino como un modelo mental de organización del código, marcando la transición entre la manipulación directa del DOM y las arquitecturas más estructuradas como MVC (Modelo–Vista–Controlador). Esta clase explora precisamente esa evolución: cómo jQuery representó el punto de inflexión entre la programación imperativa y la estructuración lógica del frontend.

De la manipulación del DOM a la modularidad

Antes de jQuery, interactuar con el DOM implicaba escribir sentencias como:

```
document.getElementById("titulo").style.color = "red";
document.querySelectorAll(".menu-item")[0].addEventListener("click", fn);
```

Cada una de estas líneas debía ajustarse según el navegador (por ejemplo, Internet Explorer usaba `attachEvent` en lugar de `addEventListener`). jQuery abstraigo esas diferencias mediante una API unificada:

```
$("#titulo").css("color", "red");
$(".menu-item").on("click", fn);
```

Este cambio simple pero poderoso tuvo tres consecuencias importantes:

1. Simplificó la curva de aprendizaje de JavaScript, especialmente en el manejo del DOM.
2. Incorporó la noción de “encapsular comportamientos”: las acciones ya no se escribían dispersas, sino dentro de bloques funcionales.
3. Introdujo la idea de modularidad, es decir, separar partes del código según su función (vista, lógica, datos), lo cual es la esencia del patrón MVC.

Sintaxis base de jQuery

Todo código jQuery se ejecuta una vez que el documento HTML está completamente cargado, garantizando que los elementos del DOM estén disponibles. La estructura fundamental es:

```
$(document).ready(function() {
    // código jQuery aquí
});
```



El símbolo `$` es un alias de la función `jQuery()`. Esta función actúa como un selector universal, similar a `document.querySelector()`, pero con capacidades adicionales para aplicar métodos de forma encadenada (chaining):

```
$("p")
  .addClass("resaltado")
  .fadeIn(400)
  .css("font-weight", "bold");
```

La encadenabilidad fue una de las innovaciones clave de jQuery, ya que fomentó la escritura de código declarativo y limpio, característica que luego adoptaron los frameworks modernos.

Tabla1. Ventajas clave de jQuery

Ventaja	Descripción
Compatibilidad universal	Unifica las diferencias entre navegadores (Chrome, Firefox, Edge, Safari).
Selectores potentes	Basados en la sintaxis de CSS, permiten acceder a elementos con precisión.
Eventos simplificados	<code>on()</code> , <code>off()</code> , <code>hover()</code> , etc., facilitan el manejo de interacciones.
Animaciones integradas	Métodos como <code>.fadeIn()</code> o <code>.slideToggle()</code> permiten efectos visuales sin CSS adicional.
AJAX optimizado	Métodos como <code>\$.getJSON()</code> o <code>\$.ajax()</code> facilitan la comunicación con servidores.
Extensibilidad	Se pueden crear plugins para extender su funcionalidad.

Selectores y manipulación del DOM

Los selectores de jQuery son tan expresivos como los de CSS:

```
$("#menu")           // Selecciona por ID
$(".activo")         // Selecciona por clase
$("li a")            // Selecciona todos los enlaces dentro de un <li>
$("input:checked")    // Selecciona todos los checkboxes activos
```

Ejemplo de manipulación del contenido y atributos:

```
$("#titulo").text("Catálogo de Productos");
$(".precio").css("color", "green");
$("img").attr("alt", "Imagen de producto");
```

Además, jQuery permite crear elementos dinámicamente:

```
$("<li>Nuevo elemento</li>").appendTo("#lista");
```

Esto introduce la noción de *renderizado dinámico*, principio que luego evoluciona en frameworks como React o Vue (DOM virtual).



Eventos y control de flujo

El sistema de eventos de jQuery es uno de los pilares de la biblioteca:

```
$("#button").on("click", function(){  
    alert("¡Botón presionado!");  
});
```

También soporta delegación de eventos, permitiendo manejar elementos creados dinámicamente:

```
$("#lista").on("click", "li", function(){  
    $(this).toggleClass("seleccionado");  
});
```

Este tipo de control refuerza la idea del Controlador en MVC, que media entre las acciones del usuario y los cambios en la Vista.

AJAX con jQuery: conexión con el modelo de datos

jQuery simplificó enormemente las solicitudes asincrónicas. Por ejemplo, para leer un archivo JSON con productos:

```
$.getJSON("productos.json", function(datos){  
    datos.forEach(p => {  
        $("#catalogo").append(`<li>${p.nombre} - ${p.precio}</li>`);  
    });  
});
```

Esta interacción marca la entrada al concepto de Modelo: el lugar donde residen los datos, independientes de la interfaz visual. Así, el flujo natural se vuelve:

Vista (HTML) ← Controlador (jQuery) ← Modelo (JSON o API)

Encadenamiento de métodos (chaining)

Una de las características más distintivas de jQuery es el encadenamiento, que permite aplicar múltiples transformaciones al mismo conjunto de elementos:

```
$("#banner")  
    .fadeOut(500)  
    .delay(300)  
    .fadeIn(500)  
    .css("border-radius", "10px");
```

Este paradigma fomenta la legibilidad y modularidad del código, principios esenciales en la transición hacia MVC.

Diagrama conceptual: evolución del pensamiento arquitectónico

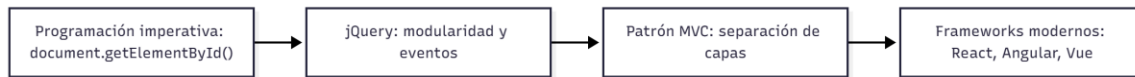


Imagen 1. Evolución del pensamiento estructurado desde jQuery hacia las arquitecturas modernas.

Fuente: Elaboración propia (2025).

Mini práctica: simulando MVC con jQuery

Estructura de carpetas:

```
app/  
├── model/productos.json  
├── view/index.html  
└── controller/app.js
```

model/productos.json

```
[  
  {"nombre": "Café", "precio": 3.5},  
  {"nombre": "Té Verde", "precio": 2.8},  
  {"nombre": "Chocolate", "precio": 4.0}  
]
```

view/index.html

```
<!DOCTYPE html>  
<html lang="es">  
  <head>  
    <meta charset="UTF-8">  
    <title>Catálogo MVC con jQuery</title>  
  </head>  
  <body>  
    <h1>Catálogo de Productos</h1>  
    <ul id="lista"></ul>  
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>  
    <script src="../controller/app.js"></script>  
  </body>  
</html>
```

controller/app.js

```
$(document).ready(function(){  
  $.getJSON("../model/productos.json", function(datos){  
    renderVista(datos);  
  });  
});  
  
function renderVista(lista){  
  $("#lista").empty();  
  lista.forEach(p => $("#lista").append(`<li>${p.nombre} - ${p.precio}</li>`));  
}
```



Modelo: contiene los datos (`productos.json`)

Vista: interfaz HTML

Controlador: gestiona la lógica (`app.js`)

Impacto de jQuery en el desarrollo moderno

Aunque muchos frameworks actuales ya no dependen de jQuery, su legado permanece:

- Introdujo el concepto de comportamiento desacoplado (el HTML deja de contener código JS incrustado).
- Promovió la estructura funcional del frontend.
- Inspiró el desarrollo de arquitecturas basadas en componentes (React, Vue).
- Consolidó el patrón MVC en el lado del cliente, antes reservado al backend.

Tabla 2. Transición: de jQuery a MVC

Etapas	Características	Ejemplo
Imperativa	Código disperso, sin modularidad	<code>document.getElementById()</code>
jQuery modular	Separación parcial entre vista y lógica	<code>\$("#btn").on("click", fn)</code>
MVC en frontend	Separación completa entre modelo, vista y controlador	jQuery + JSON
Framework moderno	Componentes reactivos y binding bidireccional	React / Angular / Vue

Conclusión

jQuery marcó el punto de inflexión entre el JavaScript manual y el JavaScript estructurado. Su impacto va más allá de la sintaxis: enseñó a pensar en términos de capas, módulos y responsabilidades, que son la base del patrón **Modelo–Vista–Controlador**. El estudiante, al dominar jQuery, no solo adquiere destreza técnica, sino que internaliza la importancia de separar la presentación (Vista), la lógica (Controlador) y los datos (Modelo), sentando las bases para los frameworks modernos.

6.2 Fundamentos y manipulación del DOM con jQuery

El **DOM (Document Object Model)** es la representación estructurada de un documento HTML o XML en forma de árbol de nodos. Cada etiqueta, texto, atributo o comentario dentro de una página se convierte en un *nodo* que puede ser accedido y modificado mediante JavaScript. Sin embargo, manipular directamente el DOM nativo puede resultar tedioso, especialmente cuando se trata de seleccionar múltiples elementos, responder a eventos o actualizar dinámicamente la interfaz.

En este contexto, jQuery simplifica la interacción con el DOM, ofreciendo una sintaxis más breve, coherente y expresiva. Sus métodos permiten recorrer, modificar, insertar y eliminar elementos de manera fluida, garantizando compatibilidad entre navegadores y

facilitando el desarrollo de aplicaciones interactivas. Dominar estos fundamentos es esencial para crear aplicaciones que respondan a las acciones del usuario y que mantengan una estructura clara y mantenible, acercando al estudiante a la lógica del patrón MVC, donde el DOM representa la *Vista*.

1. Comprendiendo el DOM con jQuery

El DOM puede imaginarse como un árbol:

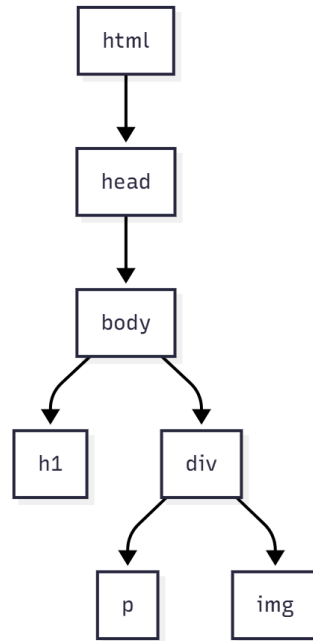


Imagen 2. Estructura jerárquica del DOM.
Fuente: elaboración propia (2025).

jQuery actúa como una interfaz que facilita la manipulación de estos nodos con comandos simples y expresivos. Por ejemplo, en lugar de escribir:

```
document.getElementById("titulo").innerText = "Catálogo actualizado";
```

Con jQuery se logra lo mismo con una sintaxis más limpia:

```
$("#titulo").text("Catálogo actualizado");
```

Esta simplicidad acelera la escritura del código y mejora su legibilidad.

2. Selectores en jQuery

Los selectores son el núcleo de jQuery. Se basan en la sintaxis de CSS y permiten identificar cualquier elemento del DOM.



Tabla 3. Selectores JQuery

Tipo de selector	Ejemplo	Descripción
Por ID	<code>\$("#menu")</code>	Selecciona el elemento con <code>id="menu"</code>
Por clase	<code>\$(".boton")</code>	Selecciona todos los elementos con clase "boton"
Por etiqueta	<code>\$("p")</code>	Selecciona todos los párrafos
Combinado	<code>\$("div p")</code>	Selecciona párrafos dentro de un div
Atributo	<code>\$("[name='usuario']")</code>	Selecciona un elemento por su atributo
Filtros	<code>\$(":first"),</code> <code>\$(":last")</code>	Selecciona el primer o último elemento de una lista

Ejemplo:

```
// Cambiar el color de los párrafos dentro de un div con clase .contenedor
$("div.contenedor p").css("color", "navy");
```

Esta expresividad permite interactuar con múltiples elementos simultáneamente, una característica esencial para desarrollar interfaces dinámicas y adaptables.

3. Manipulación del contenido

jQuery ofrece métodos para leer o modificar el contenido textual y HTML de los elementos:

Tabla 4. Métodos para leer o modificar el contenido

Método	Descripción	Ejemplo
<code>.text()</code>	Obtiene o cambia el texto interno	<code>\$("#info").text("Nuevo texto")</code>
<code>.html()</code>	Obtiene o cambia el contenido HTML	<code>\$("#panel").html("Éxito")</code>
<code>.val()</code>	Lee o cambia el valor de un input	<code>\$("#nombre").val("Juan")</code>
<code>.attr()</code> / <code>.removeAttr()</code>	Añade o elimina atributos	<code>\$("img").attr("alt", "Imagen")</code>
<code>.addClass()</code> / <code>.removeClass()</code>	Modifica clases CSS	<code>\$("p").addClass("resaltado")</code>

Ejemplo:

```
$("#bienvenida").text("¡Hola, usuario!");
$("#foto").attr("src", "avatar.png").addClass("perfil");
```

4. Inserción y eliminación de elementos

jQuery facilita crear o eliminar elementos del DOM sin recargar la página.

Creación:



```
const nuevo = $("<li>Nueva tarea</li>");
$("#lista").append(nuevo); // Agrega al final
$("#lista").prepend("<li>Primero</li>"); // Agrega al inicio
```

Eliminación:

```
$("#lista li:last").remove(); // Elimina el último elemento
$("#temporal").detach(); // Elimina sin borrar eventos asociados
```

Reemplazo:

```
$("#mensaje").replaceWith("<p>Actualizado correctamente</p>");
```

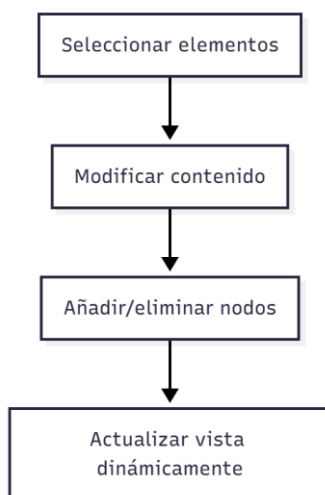


Imagen 3. Ciclo de manipulación DOM con jQuery

Fuente: elaboración propia (2025).

5. Recorrido del DOM

jQuery ofrece métodos para moverse por la jerarquía del documento:

Tabla 5. Métodos para moverse por el documento

Método	Descripción
.parent()	Devuelve el nodo padre
.children()	Devuelve los hijos directos
.find()	Busca descendientes
.siblings()	Devuelve elementos hermanos
.next() / .prev()	Devuelve el siguiente o anterior elemento

Ejemplo:

```
$("#lista li.activo").next().addClass("siguiente");
```



Estos métodos son útiles para recorrer árboles complejos y aplicar cambios específicos, lo cual se asemeja al flujo de datos en un patrón MVC, donde el controlador determina qué parte de la vista actualizar.

6. Manejo de eventos

El modelo de eventos de jQuery es uno de sus mayores aportes, ya que permite asignar funciones a las interacciones del usuario sin mezclar la lógica dentro del HTML.

```
$("#btnGuardar").on("click", function(){  
    alert("Datos guardados");  
});
```

Los eventos más comunes son:

- click, dblclick, change, submit, focus, blur, hover, keydown, keyup.

Además, es posible delegar eventos para manejar elementos creados dinámicamente:

```
$("#tabla").on("click", "tr", function(){  
    $(this).toggleClass("seleccionado");  
});
```

La delegación evita volver a asociar eventos cada vez que se inserta un nuevo nodo, optimizando el rendimiento.

7. Encadenamiento de métodos (method chaining)

Una de las características más elegantes de jQuery es el encadenamiento de métodos, que permite ejecutar varias acciones consecutivas sobre un mismo elemento:

```
$("#panel")  
    .css("background", "#f1f5f9")  
    .slideDown(400)  
    .delay(200)  
    .fadeOut(400);
```

Esto facilita la lectura del código, reduce la repetición y refuerza la lógica secuencial del Controlador dentro del patrón MVC.

8. Manipulación de atributos y propiedades

Además de `.attr()`, jQuery ofrece `.prop()` para manejar propiedades del DOM:

```
$("#check").prop("checked", true);  
$("#boton").prop("disabled", false);
```

La diferencia clave:



- `.attr()` afecta el atributo HTML original.
- `.prop()` cambia el estado en tiempo real del elemento.

Ejemplo combinado:

```
$("#foto").attr("alt", "Producto destacado");
$("#enviar").prop("disabled", true);
```

9. Integración con CSS y estilos dinámicos

jQuery puede modificar estilos directamente o agregar clases para cambiar el aspecto de los elementos:

```
$("#alerta").css({
  "background-color": "tomato",
  "color": "white",
  "padding": "10px"
});
```

Sin embargo, la mejor práctica es agregar o quitar clases CSS predefinidas:

```
$("#alerta").addClass("error").removeClass("oculto");
```

Esto mantiene la separación entre la vista (CSS) y el controlador (jQuery), en línea con el principio MVC.

10. Ejemplo práctico: Lista dinámica con jQuery

Objetivo: crear una lista de tareas donde el usuario pueda agregar y eliminar ítems.

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Lista de tareas</title>
<style>
  .tarea { margin: 5px 0; padding: 5px; border-bottom: 1px solid #ccc; }
  .completa { text-decoration: line-through; color: gray; }
</style>
</head>
<body>
  <h2>Mis tareas</h2>
  <input id="entrada" placeholder="Escribe una tarea...">
  <button id="agregar">Agregar</button>
  <ul id="lista"></ul>

  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script>
    $(document).ready(function() {
      $("#agregar").on("click", function() {
        const texto = $("#entrada").val().trim();
        if(texto) {
          $("#lista").append(`<li class="tarea">${texto}</li>`);
          $("#entrada").val("");
        }
      });
    });

    // Delegación de evento para marcar completadas
```

```
$( "#lista" ).on( "click", "li", function() {
    $( this ).toggleClass( "completa" );
} );
</script>
</body>
</html>
```

Este ejercicio ejemplifica la manipulación completa del DOM:

- Inserta nuevos nodos (`append`).
- Modifica clases dinámicamente (`toggleClass`).
- Utiliza delegación de eventos para la interacción.

11. Diagramando la lógica interna

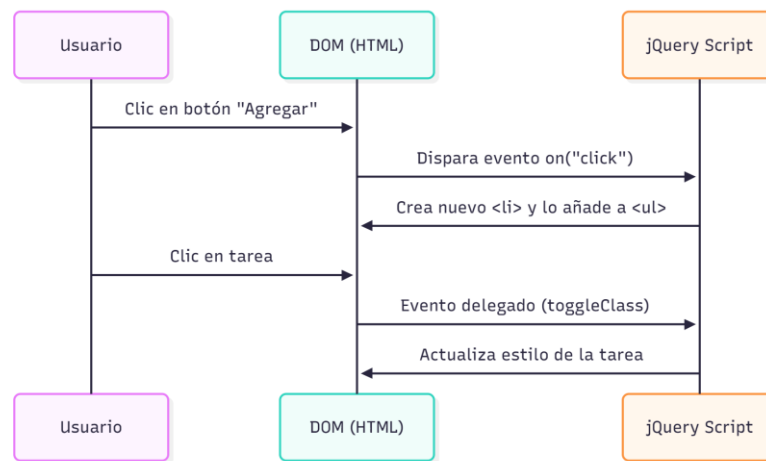


Imagen 4. Secuencia de interacción usuario–DOM–controlador con jQuery.
Fuente: elaboración propia (2025).

12. Buenas prácticas de manipulación con jQuery

1. Evitar la mezcla de lógica y presentación.
No incrustar código JS dentro de atributos HTML (`onclick`, `onchange`).
2. Usar delegación de eventos para listas o tablas dinámicas.
3. Encadenar métodos para mejorar legibilidad.
4. Evitar manipular estilos directamente.
Preferir `.addClass()` o `.removeClass()`.
5. Liberar memoria eliminando eventos con `.off()` cuando los elementos ya no se usan.



Conclusión

La manipulación del DOM con jQuery representa un punto intermedio entre la programación imperativa y la arquitectura organizada de las aplicaciones modernas. Gracias a su sintaxis simple y expresiva, los estudiantes pueden comprender cómo la vista (DOM) responde a las acciones del controlador (jQuery), generando una experiencia de usuario fluida. Estos fundamentos serán la base para introducir efectos, animaciones y posteriormente la separación de capas del patrón MVC, que consolidará la lógica del desarrollo profesional.

6.3 Efectos y animaciones

La interacción visual es un componente fundamental de las interfaces modernas. En una aplicación web, los efectos visuales no son meros adornos: cumplen una función comunicativa. Indican al usuario que una acción se ha ejecutado, que una sección ha cambiado o que un proceso se está realizando. Antes de la llegada de CSS3 y JavaScript moderno, realizar animaciones implicaba manipular manualmente estilos en intervalos (`setInterval`) o recargar imágenes. jQuery revolucionó este proceso al incorporar una API de animación declarativa, accesible y fluida, que permitía crear transiciones, efectos de aparición, desplazamientos y transformaciones con una sola línea de código.

Estos efectos constituyen una herramienta poderosa dentro de la capa de *Vista* del patrón **Modelo–Vista–Controlador (MVC)**, ya que mejoran la percepción de la aplicación sin alterar la lógica ni el modelo de datos. Además, permiten mantener la coherencia con los principios de usabilidad y accesibilidad, siempre que se usen con moderación y con sentido informativo.

1. Naturaleza de los efectos en jQuery

jQuery proporciona una capa de abstracción sobre el motor de animaciones nativo del navegador, permitiendo ejecutar transformaciones graduales en propiedades CSS. Cada animación o efecto se basa en tres fases:

1. **Inicialización:** se selecciona el elemento a animar.
2. **Transición:** se aplica un cambio de estilo de manera progresiva.
3. **Finalización:** se ejecuta una función opcional cuando la animación termina.

La sintaxis general es:

```
$(selector).efecto(velocidad, callback);
```

Donde:

- `selector` define los elementos afectados.
- `efecto` puede ser `fadeIn`, `slideDown`, `hide`, etc.

- velocidad se expresa en milisegundos (400, 600) o palabras clave ("slow", "fast").
- callback es una función que se ejecuta al finalizar la animación.

2. Efectos básicos

jQuery incluye un conjunto de efectos integrados que controlan la visibilidad y opacidad de los elementos.

Tabla 6. Efectos integrados en JQuery

Método	Descripción
<code>.show()</code> / <code>.hide()</code>	Muestra u oculta un elemento instantáneamente.
<code>.toggle()</code>	Alterna entre mostrar y ocultar.
<code>.fadeIn()</code> / <code>.fadeOut()</code>	Aparece o desaparece con una transición de opacidad.
<code>.fadeToggle()</code>	Alterna entre aparición y desaparición.
<code>.slideUp()</code> / <code>.slideDown()</code>	Desliza el contenido verticalmente.
<code>.slideToggle()</code>	Alterna entre mostrar y ocultar con desplazamiento.

Ejemplo 1:

```
$("#mostrar").on("click", function(){
    $("#panel").slideDown(400);
});

$("#ocultar").on("click", function(){
    $("#panel").slideUp(400);
});
```

Ejemplo 2:

```
$("#alternar").on("click", function(){
    $("#menu").fadeToggle(300);
});
```

Estos efectos son perfectos para secciones colapsables, menús desplegables o mensajes informativos.

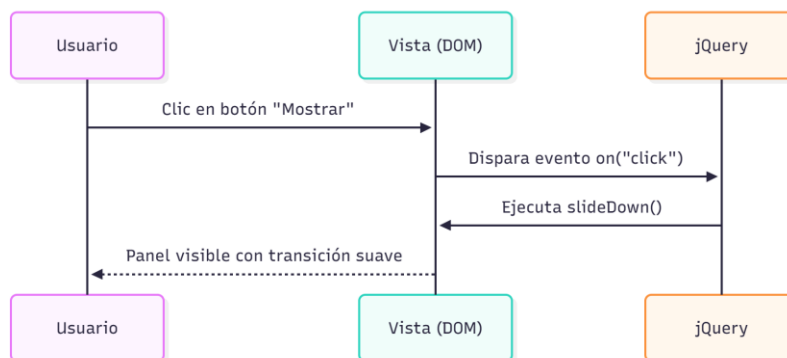


Imagen 5. Secuencia de efecto de aparición/desaparición

Fuente: elaboración propia (2025).

3. Encadenamiento de efectos

Una característica distintiva de jQuery es el encadenamiento (chaining), que permite ejecutar varios efectos o transformaciones sobre un mismo elemento de forma secuencial:

```
$("#notificacion")  
  .fadeIn(400)  
  .delay(1500)  
  .fadeOut(400);
```

En este ejemplo, el elemento aparece, se mantiene visible 1.5 segundos y luego desaparece. Este tipo de flujo visual ayuda a comunicar estados temporales, como alertas o confirmaciones de acciones.

4. Animaciones personalizadas con `.animate()`

Además de los efectos predefinidos, jQuery incluye el método `.animate()`, que permite animar propiedades CSS numéricas (por ejemplo, tamaño, posición, opacidad).

Sintaxis:

```
$(selector).animate({ propiedad: valor }, duracion, easing, callback);
```

Ejemplo:

```
$("#caja").animate({  
  left: "200px",  
  opacity: 0.7,  
  height: "200px"  
}, 800);
```

Los valores pueden expresarse en **px**, **%**, **em** o incluso con valores relativos (`+=50px`).

Esto introduce el concepto de interpolación, es decir, el cálculo automático de los valores intermedios durante la transición.

Control de velocidad y “easing”

El parámetro *easing* define la forma en que evoluciona la animación:

- `"swing"`: acelera al inicio y desacelera al final (por defecto).
- `"linear"`: mantiene velocidad constante.

Ejemplo con *callback*:



```
$("#panel").slideDown(500, function(){
    alert("La animación ha finalizado.");
});
```

El callback es una herramienta muy poderosa para encadenar efectos o ejecutar lógica posterior a una animación.

4. Control avanzado de animaciones

Tabla 7. Métodos para el control de animaciones

Método	Descripción
.stop()	Detiene una animación en curso.
.delay(ms)	Introduce una pausa entre animaciones.
.finish()	Completa instantáneamente todas las animaciones en cola.
.queue() / .dequeue()	Permiten manipular la cola de animaciones.

Ejemplo práctico:

```
$("#animar").on("click", function(){
    $("#caja")
        .animate({ left: "+=100" }, 400)
        .delay(200)
        .animate({ top: "+=50" }, 400)
        .animate({ opacity: 0.5 }, 300);
});

$("#detener").on("click", function(){
    $("#caja").stop(true, true);
});
```

En este ejemplo:

- El botón *Animar* ejecuta una serie de transiciones.
- El botón *Detener* interrumpe cualquier animación activa.

6. Animaciones combinadas con clases CSS

Aunque jQuery puede modificar estilos directamente, la buena práctica consiste en combinarlo con clases CSS para mantener la separación entre la *Vista* (CSS) y el *Controlador* (jQuery).

Ejemplo:

```
.alerta {
    background: #fee2e2;
    border-left: 5px solid #ef4444;
    padding: 10px;
    display: none;
}

.mostrar {
    display: block;
    animation: entrar 0.5s ease-in;
}

@keyframes entrar {
```



```
from { opacity: 0; transform: translateY(-10px); }  
to { opacity: 1; transform: translateY(0); }  
}  
$("#btnError").on("click", function() {  
  $(".alerta").addClass("mostrar");  
});
```

Así, jQuery se limita a gestionar cuándo ocurre el efecto, mientras que CSS define cómo debe verse.

7. Accesibilidad en animaciones (WCAG 2.1)

Según las normas WCAG, los efectos visuales deben cumplir los principios POUR (Perceptible, Operable, Comprensible y Robusto).

Algunas consideraciones clave:

1. Evitar parpadeos mayores a 3 veces por segundo.
2. Incluir un mecanismo para detener o reducir las animaciones.
3. Respetar la preferencia del usuario mediante la media query `prefers-reduced-motion`.

Ejemplo:

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation-duration: 0.001ms !important;  
    transition-duration: 0.001ms !important;  
  }  
}
```

Esto garantiza que las animaciones no afecten a usuarios con sensibilidad al movimiento.

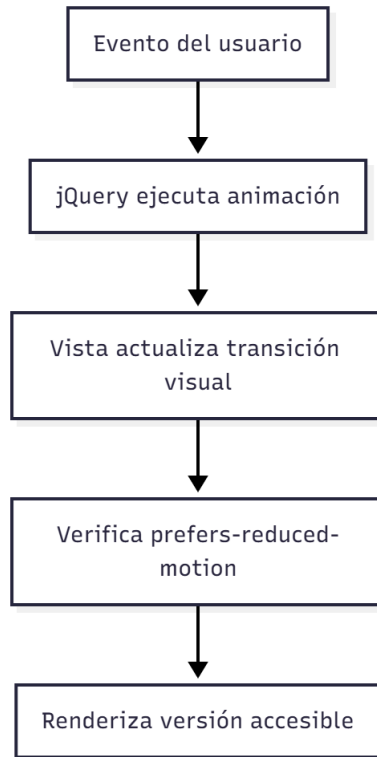


Imagen 6. Integración de animaciones accesibles

Fuente: elaboración propia (2025).

8. Mini práctica: Panel interactivo animado

Objetivo: crear un panel de información que aparezca y desaparezca con efectos combinados.

```

<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Panel Animado</title>
<style>
  #panel {
    width: 300px;
    background: #f1f5f9;
    border-radius: 10px;
    padding: 10px;
    display: none;
  }
  button { margin-top: 10px; }
</style>
</head>
<body>
  <h2>Demostración de efectos jQuery</h2>
  <button id="mostrar">Mostrar</button>
  <button id="ocultar">Ocultar</button>
  <div id="panel">
    <p>Este panel aparece y desaparece con un suave efecto.</p>
  </div>
  
```



```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<script>
  $("#mostrar").on("click", function(){
    $("#panel").fadeIn(400).animate({width: "350px"}, 300);
  });

  $("#ocultar").on("click", function(){
    $("#panel").animate({width: "300px"}, 300).fadeOut(400);
  });
</script>
</body>
</html>
```

Resultado:

- El panel se muestra con un efecto de *fadeIn*.
- Aumenta su tamaño mediante `.animate()`.
- Se oculta con una animación inversa.

9. Animaciones como comunicación visual

En diseño de interacción, los efectos deben guiar al usuario sin distraerlo. Los usos más recomendados son:

- Destacar cambios de estado (guardar, eliminar, error).
- Transiciones entre secciones o pestañas.
- Retroalimentación visual en formularios y validaciones.

Ejemplo de retroalimentación:

```
$("#form").on("submit", function(e){
  e.preventDefault();
  $("#mensaje")
    .text("Datos enviados correctamente")
    .css("color", "green")
    .fadeIn(400)
    .delay(2000)
    .fadeOut(400);
});
```

Este tipo de efecto refuerza la percepción de éxito o error sin requerir alertas intrusivas.

10. Diagrama de flujo del ciclo de animación

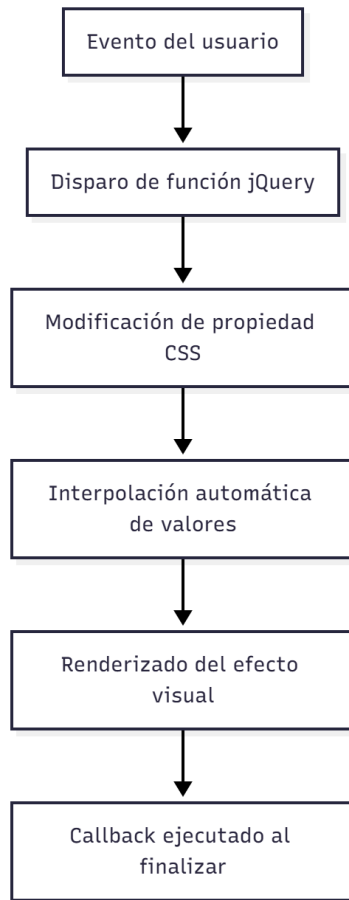


Imagen 7: Flujo de ejecución de una animación en jQuery.
Fuente: elaboración propia (2025).

11. Buenas prácticas de animación

1. Usar efectos para reforzar significado, no solo decoración. Por ejemplo, un *fadeIn* para mostrar resultados, no solo para llamar la atención.
2. Combinar efectos cortos y consistentes. Evitar transiciones largas (>1s) que ralentizan la interacción.
3. Asegurar que las animaciones sean reversibles. Si algo se muestra con *slideDown*, debe ocultarse con *slideUp*.
4. Usar `stop()` o `finish()` para evitar saturar la cola de animaciones.
5. Verificar compatibilidad con accesibilidad. Aplicar media queries para usuarios que prefieren “reduced motion”.



12. Conexión con MVC

Las animaciones se ubican exclusivamente en la capa de la Vista, actuando como una representación perceptual de los cambios producidos por el Controlador o el Modelo. Por ejemplo, cuando un producto se añade al carrito (acción del Controlador), la Vista responde con un *fadeIn* del contador o un *slideDown* de confirmación. De esta forma, el comportamiento visual no interfiere con la lógica de negocio, manteniendo la arquitectura limpia.

Conclusión

Los efectos y animaciones en jQuery son mucho más que adornos visuales: representan una herramienta de comunicación y retroalimentación en la experiencia de usuario. Comprender su funcionamiento prepara al estudiante para implementar transiciones suaves, visualmente atractivas y accesibles, que acompañen las acciones del usuario dentro del flujo MVC. Con ello, se consolida la noción de separación entre lógica, datos y presentación, y se prepara el terreno para introducir, en el siguiente punto, el patrón Modelo–Vista–Controlador (MVC) como estructura formal de una aplicación escalable.

6.4 Introducción al patrón MVC y comparación con frameworks modernos

A medida que las aplicaciones web se volvieron más interactivas, los desarrolladores descubrieron que mantener el código ordenado se volvía cada vez más difícil. En las primeras etapas, un único archivo JavaScript contenía la lógica, las manipulaciones del DOM y las solicitudes al servidor, mezclando responsabilidades. Este enfoque monolítico dificultaba el mantenimiento, la colaboración y la escalabilidad.

Para resolver este problema surgió el patrón arquitectónico MVC (Modelo–Vista–Controlador), una forma de organizar el código que separa la aplicación en tres componentes distintos pero interconectados:

- **Modelo**, que gestiona los datos y la lógica de negocio.
- **Vista**, que se encarga de la presentación y la interfaz de usuario.
- **Controlador**, que coordina las interacciones entre ambos.

Esta separación de responsabilidades no solo mejora la claridad del código, sino que permite desarrollar aplicaciones modulares, mantenibles y escalables. En este punto, el estudiante comprenderá el concepto, su flujo interno, y cómo esta estructura evolucionó hacia los frameworks modernos como React, Angular, Vue, Django o ASP.NET.

1. ¿Qué es el patrón MVC?

El patrón **Modelo–Vista–Controlador** surgió en los años 70 dentro del contexto de la programación orientada a objetos (Smalltalk) y se consolidó como una arquitectura



universal para sistemas interactivos. Su propósito es dividir las responsabilidades del software para que cada parte cumpla una función específica:

Tabla 8. Componentes MVC

Componente	Rol	Responsabilidad principal
Modelo (Model)	Gestiona los datos	Define estructuras, valida entradas, accede a bases de datos o APIs.
Vista (View)	Muestra información	Renderiza la interfaz al usuario, usando datos provenientes del modelo.
Controlador (Controller)	Coordina la lógica	Interpreta las acciones del usuario, solicita datos al modelo y actualiza la vista.

Flujo básico de MVC

1. El usuario interactúa con la *Vista* (por ejemplo, hace clic en un botón).
2. El Controlador interpreta esa acción y decide qué debe suceder.
3. El Modelo procesa la información (consulta, valida o actualiza datos).
4. El Controlador recibe los resultados del modelo y los envía a la Vista.
5. La Vista muestra la información actualizada al usuario.

Diagrama conceptual

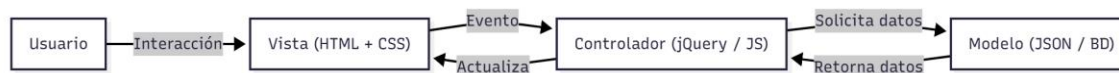


Imagen 8. Flujo de comunicación entre las capas del patrón MVC.

Fuente: elaboración propia (2025).

2. Aplicando MVC en el contexto del frontend

Aunque MVC fue inicialmente diseñado para el backend (por ejemplo, en frameworks como Ruby on Rails o ASP.NET MVC), su filosofía se adaptó también al frontend gracias a bibliotecas como jQuery, que permiten separar la lógica en capas:

- La Vista corresponde al DOM (HTML + CSS).
- El Controlador se implementa con jQuery o JavaScript, manejando eventos.
- El Modelo se representa como una fuente de datos (archivos JSON, APIs, bases locales o remotas).

Ejemplo de estructura MVC en jQuery

Estructura de carpetas:

```

app/
├── model/productos.json
├── view/index.html
└── controller/app.js
  
```

model/productos.json

```
[
  { "id": 1, "nombre": "Café Premium", "precio": 3.5 },
  { "id": 2, "nombre": "Té Verde", "precio": 2.8 },
  { "id": 3, "nombre": "Chocolate", "precio": 4.0 }
]
```

view/index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Catálogo MVC</title>
  <style>
    body { font-family: sans-serif; }
    .producto { margin: 0.5rem 0; }
  </style>
</head>
<body>
  <h1>Catálogo de productos</h1>
  <ul id="lista"></ul>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script src="../controller/app.js"></script>
</body>
</html>
```

controller/app.js

```
$(document).ready(function() {
  // Controlador: conecta la vista con el modelo
  $.getJSON("../model/productos.json", function(data) {
    renderVista(data);
  });
});

// Vista: actualiza el DOM
function renderVista(productos) {
  const lista = $("#lista");
  lista.empty();
  productos.forEach(p => lista.append(`<li class="producto">${p.nombre} -
  ${p.precio}</li>`));
}
```

La aplicación muestra productos obtenidos del modelo, gestionados por el controlador, y renderizados en la vista.

3. Beneficios de la separación MVC

Tabla 9. Separación MVC

Beneficio	Descripción
Mantenibilidad	Cada componente puede modificarse sin afectar a los demás.
Reutilización	El modelo o la vista pueden reaprovecharse en otros proyectos.
Colaboración	Los equipos pueden trabajar en paralelo: diseñadores en la vista, programadores en la lógica.
Escalabilidad	Facilita la expansión a arquitecturas más complejas (REST, SPAs).
Depuración	Los errores se localizan con mayor facilidad.

4. MVC y la modularidad en JavaScript

El patrón MVC también promueve la organización del código mediante módulos independientes, evitando el uso de variables globales y promoviendo la encapsulación. En proyectos más avanzados, esta modularidad se implementa con ES Modules, CommonJS o AMD, permitiendo importar y exportar componentes según su función.

Ejemplo simple:

```
// model.js
export const productos = [
  { nombre: "Taza", precio: 5 },
  { nombre: "Filtro", precio: 8 }
];

// controller.js
import { productos } from "../model.js";
import { render } from "../view.js";

export function iniciar(){
  render(productos);
}

// view.js
export function render(data){
  const lista = document.querySelector("#lista");
  lista.innerHTML = data.map(p => `<li>${p.nombre} - ${p.precio}</li>`).join("");
}
```

5. Comparación con frameworks modernos

A medida que las aplicaciones crecieron, el patrón MVC se expandió y se adaptó. Hoy, casi todos los frameworks modernos derivan o reinterpretan MVC de distintas maneras.

Tabla 10. Comparación entre Frameworks

Framework	Patrón base	Lenguaje	Características principales
React	MVU (Model-View-Update)	JavaScript	Basado en componentes y Virtual DOM.
Angular	MVC / MVVM	TypeScript	Estructura completa con inyección de dependencias y binding bidireccional.
Vue.js	MVVM	JavaScript	Combina reactividad y plantillas declarativas.
Svelte	Compilador reactivo	JavaScript	Genera código optimizado sin Virtual DOM.
Django	MTV (Model-Template-View)	Python	MVC orientado al backend con plantillas HTML.
ASP.NET MVC	MVC clásico	C#	Integración con Razor Pages y lógica de servidor.

Evolución histórica

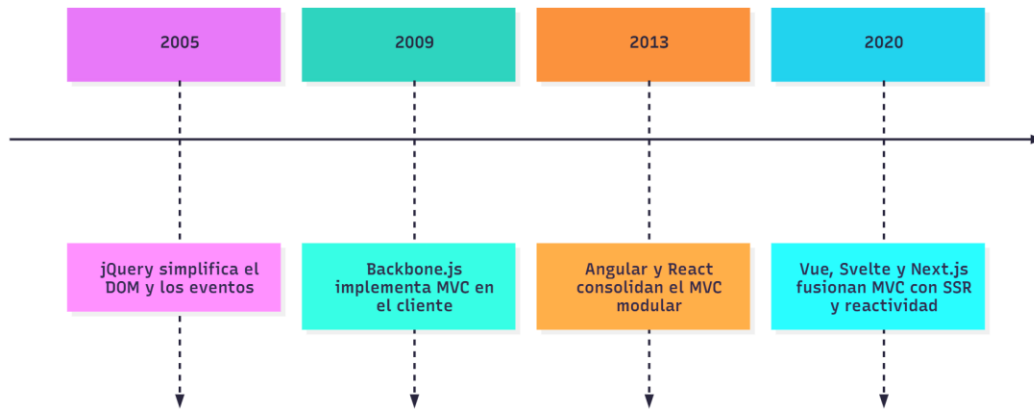


Imagen 9. Línea de tiempo de la evolución del patrón MVC hacia frameworks modernos. Fuente: elaboración propia (2025).

6. MVC en aplicaciones modernas: flujo interactivo

Los frameworks actuales mantienen la esencia del MVC pero integran conceptos como:

- **Binding de datos:** sincronización automática entre modelo y vista.
- **Componentes:** pequeñas unidades reutilizables que encapsulan lógica y presentación.
- **Virtual DOM:** mejora el rendimiento al renderizar solo los cambios.
- **Reactive programming:** las vistas responden a cambios del modelo en tiempo real.

Ejemplo conceptual en React:

```
function Producto({ nombre, precio }) {
  return <li>{nombre} - ${precio}</li>;
}
```

Este componente combina las tres capas (modelo, vista y controlador) de forma declarativa, pero la lógica sigue siendo la misma: los datos (modelo) se representan en la interfaz (vista) mediante un flujo controlado (controlador).

7. MVC y APIs REST

En el backend, MVC también se relaciona con las API REST, donde cada capa tiene un propósito específico:

- **Modelo:** define las entidades y reglas de negocio.
- **Controlador:** expone endpoints (/productos, /usuarios).



- **Vista:** devuelve las respuestas en formato JSON o HTML.

Ejemplo conceptual (Node.js + Express):

```
app.get("/productos", async (req, res) => {  
  const productos = await Producto.find();  
  res.json(productos);  
});
```

El cliente jQuery o frontend MVC consume estos datos mediante `$.ajax()` o `fetch()`, manteniendo la comunicación clara entre las capas.

8. Mini práctica: simulación MVC con actualización dinámica

Objetivo: implementar una mini aplicación que actualice la vista cuando cambian los datos del modelo.

Estructura:

```
mvc-demo/  
├── model/datos.json  
├── view/index.html  
└── controller/app.js
```

model/datos.json

```
[  
  { "nombre": "Laptop", "precio": 950 },  
  { "nombre": "Mouse", "precio": 25 }  
]
```

view/index.html

```
<!DOCTYPE html>  
<html lang="es">  
<head>  
<meta charset="UTF-8">  
<title>Demo MVC dinámica</title>  
</head>  
<body>  
  <h2>Inventario</h2>  
  <ul id="lista"></ul>  
  <button id="actualizar">Actualizar datos</button>  
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>  
  <script src="../controller/app.js"></script>  
</body>  
</html>
```

controller/app.js

```
$(document).ready(function() {  
  cargarDatos();  
  
  $("#actualizar").on("click", function() {  
    cargarDatos();  
  });  
});
```



```
});

function cargarDatos(){
  $.getJSON("../model/datos.json", function(datos){
    renderVista(datos);
  });
}

function renderVista(lista){
  const ul = $("#lista");
  ul.empty();
  lista.forEach(item => ul.append(`<li>${item.nombre} - ${item.precio}</li>`));
}
```

Cada vez que se presiona “Actualizar”, la vista se reconstruye a partir del modelo. Este flujo simple refleja los principios del MVC: control de eventos, acceso a datos y actualización de la interfaz.

9. Tabla comparativa: MVC clásico vs moderno

Tabla 11. Comparativa: MVC clásico vs moderno

Criterio	MVC tradicional (jQuery / Express)	MVC moderno (React / Angular / Vue)
Separación física	Archivos separados (HTML, JS, JSON)	Componentes integrados
Flujo de datos	Unidireccional (Controlador → Vista)	Bidireccional o reactivo
Renderizado	Directo en DOM real	Virtual DOM / reactividad
Escalabilidad	Limitada	Alta
Enfoque	Estructural	Declarativo y funcional

10. Conclusión

El patrón Modelo–Vista–Controlador es una de las bases conceptuales más importantes en el desarrollo de software moderno. Permite separar responsabilidades, mantener coherencia entre capas y fomentar la colaboración multidisciplinaria. En el contexto del frontend, jQuery sirve como una herramienta introductoria para comprender este flujo de trabajo, donde los datos (modelo) se gestionan de forma independiente de la vista (DOM), y los eventos (controlador) actúan como puente entre ambos. Comprender este patrón prepara al estudiante para trabajar con frameworks contemporáneos, donde el MVC evoluciona hacia estructuras reactivas, modulares y escalables, sin abandonar su esencia: organizar el código para mantener el control sobre la complejidad.

6.5 Persistencia y almacenamiento de datos en el modelo (solo lectura del lado del cliente)

En el desarrollo de aplicaciones web modernas, los datos son el eje central de toda funcionalidad. Sin una adecuada gestión de la información, las aplicaciones se vuelven efímeras y los usuarios pierden confianza en el sistema. Dentro del patrón Modelo–Vista–Controlador (MVC), el modelo (Model) es el encargado de gestionar el acceso,



estructura y coherencia de los datos. En esta etapa del curso, se abordará la persistencia del lado del cliente, enfocándose únicamente en la lectura y consulta de fuentes de información estáticas o accesibles directamente desde el navegador.

La persistencia en el cliente es fundamental para aplicaciones que deben operar en modo offline o consumir información sin depender de un servidor remoto. Los principales mecanismos que estudiaremos son:

1. Archivos planos: JSON, CSV y XML.
2. Bases embebidas en memoria: SQLite (sql.js).
3. Lectura desde bases de datos en la nube: Firebase Firestore (solo lectura).

Estos métodos permiten construir modelos robustos que entregan datos estructurados a las vistas, garantizando independencia y modularidad en el diseño del sistema.

1. Conceptos clave: almacenamiento y persistencia en cliente

Almacenamiento es el proceso de guardar información de manera temporal o permanente, ya sea en memoria, archivos o bases locales.

Persistencia, en cambio, implica que los datos sobrevivan más allá de la ejecución del programa o de la sesión del navegador.

En el contexto del patrón MVC aplicado al cliente, el modelo debe cumplir con cuatro funciones esenciales:

1. Definir la estructura de los datos (campos, tipos y relaciones básicas).
2. Leer información desde fuentes accesibles por el navegador.
3. Transformar los datos en estructuras uniformes para el controlador.
4. Mantener coherencia e integridad en la información mostrada por la vista.

La lectura local de datos estáticos (JSON, CSV, XML) permite al estudiante comprender cómo el modelo abstrae la fuente de datos, asegurando independencia respecto a la vista y el controlador.

2. Archivos estáticos: JSON, CSV y XML

Los archivos planos son las fuentes más simples de persistencia en cliente. Se encuentran en la carpeta del proyecto y se acceden mediante peticiones **fetch()** o librerías del navegador. Son útiles para prototipos, pruebas de interfaz y simulaciones de bases de datos.

2.1 JSON: estructura jerárquica

El formato JSON (JavaScript Object Notation) representa los datos como pares *clave–valor*, fácilmente interpretables por JavaScript.



Ejemplo de estructura:

```
[
  {"id":1, "nombre":"Café Orgánico", "precio":3.5},
  {"id":2, "nombre":"Té Verde", "precio":2.8}
]
```

Lectura básica con `fetch`:

```
fetch("data/productos.json")
  .then(res => res.json())
  .then(datos => {
    datos.forEach(p => {
      document.querySelector("#lista").innerHTML += `<li>${p.nombre} - $${p.precio}</li>`;
    });
  });
```

Este método no requiere dependencias externas y permite entregar al controlador un arreglo de objetos con campos estandarizados (nombre, precio, categoría).

2.2 CSV: *formato tabular*

El formato CSV (Comma-Separated Values) es ideal para conjuntos de datos estructurados en filas y columnas.

Ejemplo:

```
nombre,precio,categoria
Café Orgánico,3.5,Bebida
Té Verde,2.8,Bebida
```

Lectura con **PapaParse**:

```
<script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.2/papaparse.min.js"></script>
<script>
Papa.parse("data/productos.csv", {
  download: true,
  header: true,
  complete: function(results){
    console.table(results.data);
  }
});
</script>
```

CSV permite interoperabilidad con sistemas externos y hojas de cálculo, facilitando su uso como fuente de datos simple para el modelo.

2.3 XML: *formato jerárquico basado en etiquetas*

XML (eXtensible Markup Language) es un formato estructurado, similar a HTML, útil para representar relaciones jerárquicas entre datos.

Ejemplo:

```
<productos>
  <producto>
    <nombre>Café Orgánico</nombre>
    <precio>3.5</precio>
  </producto>
  <producto>
    <nombre>Té Verde</nombre>
    <precio>2.8</precio>
  </producto>
</productos>
```

Lectura con DOMParser:

```
fetch("data/productos.xml")
  .then(r => r.text())
  .then(xml => {
    const doc = new DOMParser().parseFromString(xml, "application/xml");
    doc.querySelectorAll("producto").forEach(p => {
      console.log(p.querySelector("nombre").textContent);
    });
  });
```

XML sigue siendo útil cuando se requiere compatibilidad con sistemas legados o datos anidados complejos.

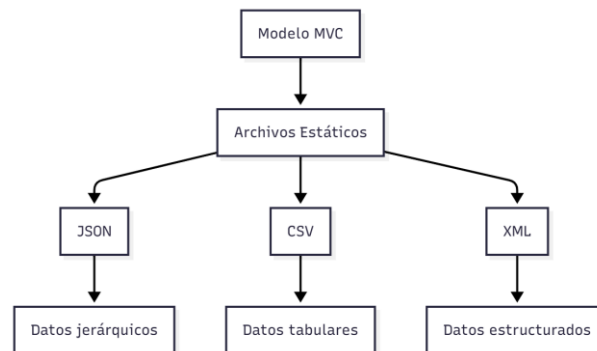


Imagen 10. Fuentes estáticas de datos del lado del cliente

Fuente: elaboración propia (2025).

3. SQLite con sql.js: persistencia relacional en cliente

SQLite es una base de datos relacional ligera que, mediante sql.js, puede ejecutarse directamente en el navegador usando WebAssembly. Permite realizar consultas SQL sin instalar un motor de servidor.

Ejemplo básico:

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/sql.js/1.6.2/sql-wasm.js"></script>
<script>
initSqlJs().then(SQL => {
  const db = new SQL.Database();
  db.run("CREATE TABLE productos (nombre TEXT, precio REAL);");
});
```

```
db.run("INSERT INTO productos VALUES ('Café', 3.5), ('Té Verde', 2.8);");
const res = db.exec("SELECT * FROM productos;");
console.table(res[0].values);
});
</script>
```

Este tipo de persistencia no modifica archivos físicos, pero permite experimentar con estructuras SQL y comprender cómo el modelo organiza los datos para entregarlos a la vista.

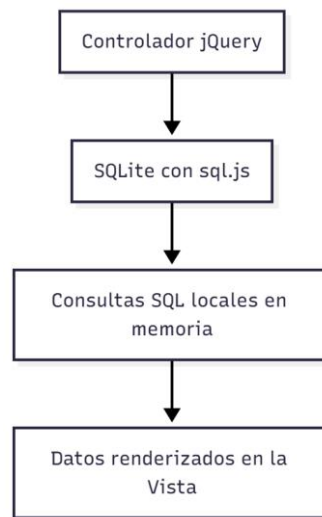


Imagen 11. Flujo de lectura de datos con SQLite en el navegador
Fuente: elaboración propia (2025).

4. Firebase Firestore: lectura de datos en la nube

Firebase Firestore es una base de datos NoSQL en la nube que puede integrarse directamente con aplicaciones del lado del cliente. En esta etapa, se utiliza exclusivamente en modo lectura, sin operaciones de escritura ni autenticación.

Ejemplo:

```
<script type="module">
  import { initializeApp } from "https://www.gstatic.com/firebasejs/10.7.0/firebase-app.js";
  import { getFirestore, collection, getDocs } from
  "https://www.gstatic.com/firebasejs/10.7.0/firebase-firestore.js";

  const config = { apiKey: "...", projectId: "miProyecto" };
  const app = initializeApp(config);
  const db = getFirestore(app);

  const productos = await getDocs(collection(db, "productos"));
  productos.forEach(doc => console.log(doc.data()));
</script>
```

Esta integración permite que el modelo consulte datos remotos y los entregue a la vista sin necesidad de backend propio.



Ventajas de Firestore (modo lectura):

- Datos actualizados en tiempo real.
- Sincronización automática del cliente.
- Ideal para dashboards y catálogos.

5. Comparativa general de tecnologías de lectura en cliente

Tabla 12. Comparativa tecnologías lectura en el cliente

Tecnología	Tipo	Persistencia	Estructura	Ideal para
JSON	Archivo plano	Estática	Jerárquica (clave-valor)	Catálogos simples
CSV	Archivo plano	Estática	Tabular	Listas de datos
XML	Archivo plano	Estática	Jerárquica con etiquetas	Interoperabilidad
SQLite (sql.js)	Relacional embebida	En memoria	Tablas con esquema	Simulación SQL
Firestore	NoSQL nube	Dinámica	Documentos y colecciones	Lectura en tiempo real

Fuente: elaboración propia (2025).

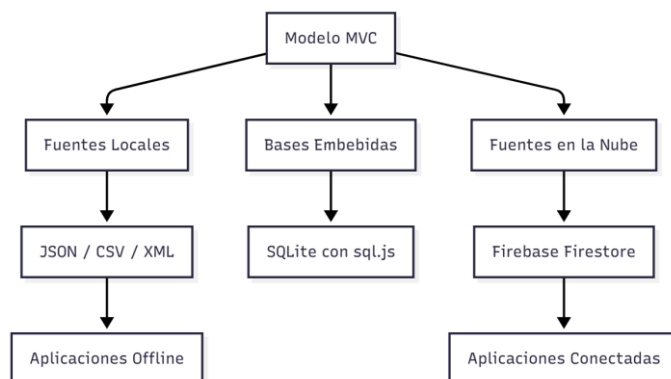


Imagen 12. Mapa conceptual del modelo de lectura de datos en cliente

Fuente: elaboración propia (2025).

6. Buenas prácticas del modelo de lectura

1. Validar la estructura del archivo antes de procesarlo.
2. Normalizar los datos: unificar nombres de campos para que la vista los interprete de manera consistente.
3. Evitar dependencias de escritura: centrarse en lectura y transformación de datos.
4. Separar las funciones de cada fuente de datos (un método por tipo: JSON, CSV, XML, etc.).
5. Controlar errores de red o parseo mediante bloques `try...catch`.



6. Mantener coherencia entre las estructuras de salida del modelo y los requerimientos de la vista.
7. Documentar las rutas y estructuras esperadas en cada fuente.

7. Conclusión

La lectura y gestión de datos del lado del cliente representan la base del modelo en el patrón MVC. A través del uso de formatos estáticos como JSON, CSV y XML, así como de bases embebidas como SQLite (sql.js) y servicios de lectura remota como Firebase Firestore, el estudiante aprende a estructurar y acceder a datos de manera coherente, segura y modular.

Esta comprensión prepara el camino para la siguiente etapa: la persistencia dinámica y sincronización en entornos cliente–servidor, donde el modelo no solo leerá datos, sino también los creará, modificará y sincronizará con bases de datos remotas.

En síntesis, dominar la lectura y representación de datos locales es el primer paso para comprender el funcionamiento lógico del modelo y su rol como núcleo de la aplicación web moderna.

Referencias citadas en la Clase 6.

- Mozilla Developer Network (MDN). (2025). Document Object Model (DOM) – Guía completa. Recuperado de https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model
- jQuery Foundation. (2024). jQuery API Documentation. Recuperado de <https://api.jquery.com/>
- Google Firebase. (2025). Cloud Firestore Documentation. Recuperado de <https://firebase.google.com/docs/firestore>
- SQLite.org. (2024). SQLite Home Page. Recuperado de <https://www.sqlite.org/index.html>
- MongoDB Inc. (2024). MongoDB Manual. Recuperado de <https://www.mongodb.com/docs/>
- PapaParse.js. (2025). Fast and powerful CSV parser for JavaScript. Recuperado de <https://www.papaparse.com/>.

Definición de términos

- MVC (Modelo–Vista–Controlador): Patrón arquitectónico que separa la aplicación en tres capas: el Modelo gestiona los datos, la Vista se encarga de la interfaz de usuario, y el Controlador coordina la lógica entre ambas. Facilita la mantenibilidad y escalabilidad del software.
- Persistencia de datos: Capacidad de una aplicación para conservar información de forma permanente, de modo que pueda recuperarse incluso después de cerrar o



reiniciar el sistema. Se logra mediante archivos (JSON, CSV) o bases de datos (SQLite, MongoDB, Firebase).

- NoSQL (Not Only SQL): Paradigma de bases de datos no relacionales que almacena información en estructuras flexibles como documentos JSON, colecciones o grafos, priorizando la escalabilidad y la velocidad sobre la rigidez del esquema.
 - Firebase Firestore: Servicio en la nube de Google para el almacenamiento de datos en tiempo real. Permite sincronización inmediata entre el cliente y el servidor, ideal para aplicaciones colaborativas.
 - jQuery Biblioteca de JavaScript que simplifica la manipulación del DOM, la gestión de eventos, las animaciones y las peticiones AJAX, promoviendo una sintaxis clara y la separación entre lógica y presentación.
- Recursos de profundización

Recurso: *Profundizacion1_clase6.docx*

ACTIVIDAD DE EVALUACIÓN:

INSTRUCCIONES	Responde las siguientes preguntas de opción múltiple relacionadas con los temas abordados en la Clase. Selecciona la alternativa correcta en cada caso. Al finalizar, revisa la retroalimentación para reforzar tu comprensión.				
Titulo	jQuery y transición a arquitectura MVC				
Dificultad		Baja	X	Media	Alta
RDAs Evaluados:	X	RDA 1			
		RDA 2			
		RDA 3			
Secciones Evaluadas		Clase: 6	jQuery y transición a arquitectura MVC		
PREGUNTA 1	¿Cuál de las siguientes opciones describe correctamente el propósito del patrón MVC?				
CORRECTA	Una estructura de software que separa la lógica, los datos y la presentación.				



Incorrecta	Un modelo de bases de datos relacional con tres tablas principales.
Incorrecta	Un sistema de almacenamiento basado en colecciones de documentos.
Incorrecta	Un algoritmo para crear efectos visuales en el navegador.
Retroalimentación de las respuestas	El patrón MVC (Modelo-Vista-Controlador) organiza el código en tres capas independientes para mejorar la mantenibilidad: <i>Modelo</i> (datos), <i>Vista</i> (interfaz) y <i>Controlador</i> (lógica).
PREGUNTA 2	¿Qué diferencia fundamental existe entre SQLite y MongoDB?
CORRECTA	SQLite es relacional y guarda datos en tablas; MongoDB es NoSQL y almacena documentos JSON.
Incorrecta	SQLite usa documentos JSON, mientras que MongoDB usa tablas relacionales.
Incorrecta	Ambas son bases de datos en la nube con esquemas fijos.
Incorrecta	MongoDB requiere servidor local y SQLite no.
Retroalimentación de las respuestas	SQLite sigue el modelo relacional clásico (tablas, filas y columnas), mientras que MongoDB utiliza documentos JSON con estructura flexible.
PREGUNTA 3	¿Qué formato de datos es más adecuado para prototipos o almacenamiento ligero en el navegador?
CORRECTA	JSON
Incorrecta	MongoDB
Incorrecta	CSV
Incorrecta	SQLite



Retroalimentación de las respuestas	El formato JSON es ideal para representar estructuras jerárquicas y es nativo en JavaScript, lo que lo hace perfecto para prototipos y persistencia ligera.
PREGUNTA 4	¿Qué ventaja principal ofrece Firebase Firestore respecto a los demás sistemas de almacenamiento?
CORRECTA	Sincroniza datos en tiempo real en la nube sin necesidad de servidor propio.
Incorrecta	Funciona sin conexión a Internet.
Incorrecta	Permite ejecutar consultas SQL directamente.
Incorrecta	Solo puede almacenar archivos CSV y JSON.
Retroalimentación de las respuestas	Firebase Firestore es una base de datos NoSQL en la nube que mantiene sincronización automática entre cliente y servidor, facilitando la persistencia en tiempo real.

Enlaces externos recomendados

Título del enlace relacionado: Documentación oficial de jQuery

Descripción del enlace relacionado: Recurso oficial con la API completa de jQuery, ejemplos prácticos y documentación detallada sobre manipulación del DOM, eventos y efectos.

Enlace: <https://api.jquery.com/>

Título del enlace relacionado: Introducción a Firebase Firestore

Descripción del enlace relacionado: Guía oficial de Google para implementar Firestore, con ejemplos de lectura, escritura y sincronización de datos en tiempo real.

Enlace: <https://firebase.google.com/docs/firestore>

Título del enlace relacionado: SQLite en el navegador con sql.js

Descripción del enlace relacionado: Implementación de SQLite en WebAssembly que permite ejecutar bases de datos locales directamente en el navegador.

Enlace: <https://sql.js.org/>

Título del enlace relacionado: MongoDB University

Descripción del enlace relacionado: Plataforma de aprendizaje gratuito sobre MongoDB, con cursos introductorios sobre consultas, colecciones y diseño de modelos

de datos NoSQL.

Enlace: <https://learn.mongodb.com/>